

Semantic-Aware Multi-LoRA Orchestration in Edge Federation: Hierarchical Multi-Timescale Optimization under QoE Constraints

Guan-Lin Chen , *Student Member, IEEE*, Yao Chiang , *Member, IEEE*, and Hung-Yu Wei , *Fellow, IEEE*

Abstract—The rapid scaling of Large Language Models (LLMs) has driven the adoption of Multi-LoRA as a lightweight mechanism for model customization. However, deploying Multi-LoRA in edge federations introduces significant challenges in operational expenditure (OPEX) and latency efficiency. In this paper, we introduce a semantic-aware Multi-LoRA serving framework for edge federations that jointly optimizes OPEX and Quality of Experience (QoE). Specifically, we propose a semantic compatibility model based on behavioral embeddings to enable high-fidelity LoRA substitution, thereby reducing cold-start latency and redundant cross-region data transfers. We further develop a hierarchical multi-timescale architecture that separates long-term model provisioning from real-time request dispatching, allowing the system to adapt to dynamic workload skewness under data sovereignty constraints. Additionally, we design a Lyapunov-based dynamic dispatching algorithm to jointly optimize cross-cluster routing, node auto-scaling, and VRAM execution modes in real time. Extensive discrete-event simulations based on real-world Microsoft Azure Functions traces demonstrate that the proposed framework significantly mitigates the batch-squeezing effect and consistently outperforms state-of-the-art baselines in steady-state throughput, long-term OPEX reduction, and 95th-percentile Time-To-First-Token (TTFT) Service Level Objective (SLO) compliance.

Index Terms—Edge computing, Resource management, Multi-LoRA serving, Service orchestration, Quality of Experience (QoE), Large language models (LLMs)

I. INTRODUCTION

In recent years, the rapid adoption of large language models (LLMs) has significantly expanded their service scale [1]. The increasing regional heterogeneity and multi-domain user demands have fostered the development of multi-agent systems and multi-expert architectures [2]. However, conventional Retrieval-Augmented Generation (RAG) [3] introduces privacy concerns due to centralized data storage, while its latency and VRAM overhead scale linearly with knowledge base size, limiting scalability. To improve efficiency, lightweight customization techniques such as Multi-LoRA (Low-Rank Adaptation) [4] have been proposed. Nevertheless, cloud-centric inference suffers from high operational costs, network latency, and privacy constraints, motivating edge deployment [5], [6]. Deploying Multi-LoRA at the edge remains challenging due to limited GPU VRAM, leading to resource contention and the “batch-squeezing effect,” where concurrent LoRA adapters reduce effective batch size [7]. Meanwhile, spatio-temporal workload skewness further complicates the joint optimization

of resource utilization, Quality of Experience (QoE), and operational cost [8].

To address these challenges, prior work has explored multiple directions. For single-node or cluster-level optimization, customized kernels and memory paging reduce fragmentation [9], [10], while cross-node migration improves throughput [11]. For elastic deployment, serverless architectures enable dynamic scaling [12], and predictive placement reduces cold-start latency [13]–[15]. For LoRA selection, Top- k strategies [16] and dynamic weight merging [17] balance accuracy and latency. In scheduling, multi-priority queues [18], learning-based approaches [19]–[21], and dynamic VRAM management [11] mitigate resource contention.

Despite these efforts, several limitations persist in edge federation scenarios. Existing adapter selection relies on label matching or heuristic Top- k methods, which fail to capture semantic similarity and often lead to redundant transfers, cold starts, and quality degradation [22]. Meanwhile, VRAM management lacks a global perspective, limiting its effectiveness under multi-tenant contention. In addition, existing predictive techniques are often limited to placement decisions and lack integrated predictive provisioning to handle transient workload dynamics. Cross-region orchestration further overlooks data sovereignty, storage constraints, and predictive provisioning requirements, limiting both compliance and efficiency in geographically distributed edge environments. Finally, throughput-oriented over-provisioning leads to excessive energy consumption without jointly optimizing QoE and operational costs.

To overcome these challenges, we propose a semantic-aware edge federated Multi-LoRA serving architecture. The design leverages behavioral embeddings for semantic compatibility evaluation and integrates Lyapunov optimization within a hierarchical multi-timescale framework to jointly optimize QoE and operational costs. The main contributions are summarized as follows:

1) *Semantic-aware Multi-LoRA Orchestration*: We design a semantic-aware orchestration mechanism that enables compatibility-driven LoRA substitution via behavioral embeddings under QoE-constrained semantic similarity. This mechanism is supported by adaptive merged/unmerged execution, which improves GPU efficiency and reduces cold-start, cross-region transfer, and batch-squeezing overhead.

2) *Predictive Multi-cluster Architecture with Data Sovereignty*: We propose a geographically distributed multi-cluster orchestration framework that integrates predictive LoRA provisioning with data-sovereignty enforcement and

TABLE I: Comparison of Related Works in Multi-LoRA Inference Systems

System	Deployment Scale	LoRA Flexibility	LoRA Execution Mode	Predictive LoRA Provisioning	Data Sovereignty	Edge Storage Constraint	Scheduling Strategy	Optimization Goal
[10]	Cluster	×	Unmerged	×	×	×	Load-aware	Throughput
[23]	Cluster	×	Unmerged	×	×	×	Resource-aware	Throughput
[24]	Single-Node	×	Unmerged	×	×	×	Load-aware	Throughput
[25]	Single-Node	×	Unmerged	×	×	×	×	Throughput
[26]	Cluster	×	Unmerged	×	×	×	Resource-aware	Throughput
[16]	Single-Node	Top- <i>k</i>	Unmerged	×	×	×	×	Throughput
[12]	Serverless	×	Unmerged	×	×	×	Load-aware	TTFT / Cost
[27]	Cluster	×	Unmerged	×	×	×	Resource-aware	TTFT
[28]	Single-Node	×	Unmerged	×	×	×	Resource-aware	TTFT
[17]	Single-Node	Weighted merge	Unmerged	×	×	×	Load-aware	Quality / zero-shot
[18]	Cluster	×	Unmerged	×	×	×	Load-aware	TTFT / Throughput
[13]	Serverless	×	Unmerged	✓	×	×	Resource-aware	TTFT
[9]	Single-Node	×	Unmerged	×	×	×	×	Throughput
[11]	Cluster	×	Hybrid	×	×	×	Resource-aware	Throughput
Ours	Multi-Cluster	Semantic substitute	Hybrid	✓	✓	✓	QoE-cost-aware	Cost / TTFT / QoE

edge storage constraints, enabling compliant and cost-efficient service management across resource-limited edge clusters.

3) *Joint Cost Optimization under QoE Constraints*: We develop a Lyapunov-based optimization framework with dynamic auto-scaling that minimizes end-to-end operational cost (compute, network, storage) while maintaining tail-latency QoE compliance.

The remainder of this paper is organized as follows. Section II reviews related work. Section III presents the system model and architecture. Section IV formulates the problem and its multi-timescale decomposition. Section V describes the proposed methods. Section VI evaluates performance. Section VII concludes the paper and outlines future directions.

II. RELATED WORK

Existing studies on Multi-LoRA inference systems have investigated resource management and performance optimization from multiple perspectives. We categorize related work into four aspects: execution engine design, deployment architectures, scheduling strategies, and adapter flexibility, and analyze their limitations in multi-cluster edge environments.

A. Multi-LoRA Execution Engine Designs

To address VRAM fragmentation and memory bottlenecks in multi-tenant LLM inference, prior work focuses on execution engine optimization. S-LoRA [9] introduces unified paging to decouple KV cache and adapter storage, reducing fragmentation. PUNICA [10] proposes customized kernels (SGMV) to accelerate heterogeneous matrix operations for parallel requests. At the system level, LoRA-Inlaid [24] combines task grouping with quantization, while dLoRA [11] improves utilization via cross-node migration.

However, these approaches primarily target single-node or single-cluster settings. They lack a global resource management perspective and cannot effectively coordinate resource allocation across distributed nodes. As a result, they are insufficient to handle dynamic multi-tenant contention and fail to mitigate the batch-squeezing effect under geographically distributed edge deployments, where resource heterogeneity and workload skewness are more pronounced.

B. Dynamic Deployment and Predictive Provisioning

To handle bursty workloads and cold-start latency, existing studies explore elastic deployment via reactive serverless designs and proactive provisioning. ServerlessLoRA [12] enables elastic scaling through shared base models, while Predictive-LoRA [13] leverages traffic prediction to reduce cold-start. JointSerLoRA [26] further introduces a two-tier architecture for coordinated caching and request grouping.

However, these approaches are primarily designed for centralized or single-cluster settings and lack support for edge federation. Serverless designs rely on flat control, limiting cross-cluster coordination, while predictive methods are confined to local environments. Moreover, they do not consider data sovereignty or edge storage constraints, and cannot jointly optimize LoRA provisioning across regions. This limits their applicability in multi-cluster edge systems.

C. Scheduling Strategies and Optimization Goals

Scheduling approaches for Multi-LoRA systems mainly aim to improve performance via load balancing and latency optimization. LoRAServe [23] proposes load-aware placement for adapter heterogeneity, while Chameleon [18] uses multi-priority queues and adaptive caching. Latency-driven methods, such as Toppings (CaraServe) [27], optimize TTFT via CPU-assisted prefill, and FastLibra [28] jointly optimizes caching and generation using a dependency tree.

However, these methods are largely designed for cloud-centric environments, where resource costs are assumed static and abundant. As a result, they primarily optimize metrics such as TTFT or throughput, while overlooking dynamic operational costs, including energy consumption and cross-region data transfer. This limitation reduces their applicability in edge scenarios, where resource constraints and cost-awareness are critical for sustainable system operation.

D. Adapter Flexibility for Environment Adaptation

Traditional Multi-LoRA systems require exact adapter matching, leading to VRAM contention and I/O overhead in resource-limited edge environments. Recent work introduces flexible mechanisms to relax this constraint. EdgeLoRA [16] adopts Top-*k* semantic routing, LoRA on the Go (LoGo) [17]

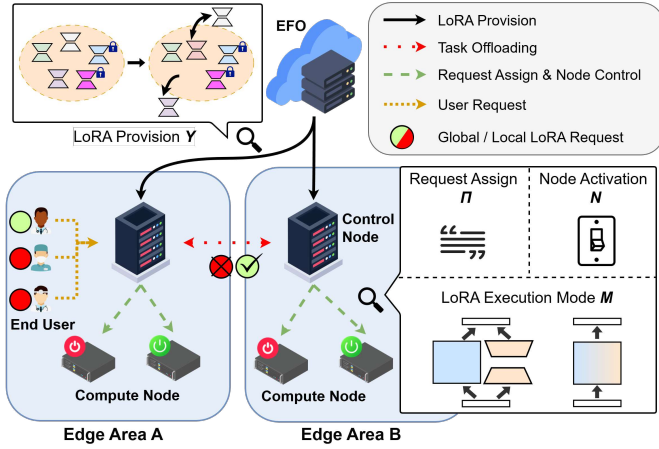


Fig. 1: Overview of the three-tier edge inference system architecture.

supports training-free dynamic merging, and MixLoRA [25] models adapters as a Mixture-of-Experts to reduce cold-start overhead.

However, these approaches rely on heuristic selection or unverified merging strategies, lacking explicit guarantees on semantic quality. While they improve latency, they do not rigorously evaluate compatibility between adapters and tasks. Consequently, such substitutions may introduce semantic inconsistency, output distortion, or hallucination, highlighting the need for principled semantic-aware mechanisms with quality assurance.

Beyond the tabular comparison, Table I reveals a structural gap in existing Multi-LoRA serving systems. Most prior designs focus on execution efficiency, scheduling, or deployment in isolation, and typically assume exact adapter matching and unconstrained service placement. Even when predictive provisioning or adapter flexibility is considered, these mechanisms are usually limited to single-node or single-cluster settings and do not explicitly account for data sovereignty or edge-side LoRA storage constraints. As a result, existing systems do not jointly support multi-cluster edge deployment, semantic-compatible substitution, predictive LoRA provisioning, sovereignty-aware placement, storage-constrained orchestration, and QoE-cost-aware hybrid execution. This motivates our unified framework, in which semantic substitution is elevated from a heuristic acceleration technique to a system-level orchestration primitive coordinated with predictive provisioning and real-time dispatching in edge federation.

III. SYSTEM MODEL

We consider a hierarchical multi-cluster edge inference system for managing Multi-LoRA services under strict Service Level Objective (SLO) and data sovereignty constraints. As illustrated in Fig. 1, the system supports a closed-loop management workflow that includes LoRA provisioning, cross-cluster task offloading, request assignment, node activation, and VRAM execution-mode control. At a high level, long-term provisioning determines which LoRA adapters are available at each edge area, while short-term scheduling decides how

incoming requests are routed and executed under latency, resource, and sovereignty constraints. Depending on feasibility and SLO compliance, a user request can be served locally, offloaded to an eligible remote cluster, or rejected with a penalty. This architecture coordinates model placement and inference scheduling across multiple edge areas to support heterogeneous workloads while minimizing operational costs. We next describe the network architecture, request model, LoRA inference design, VRAM-constrained execution modes, latency model, and cost formulation.

The system operates over two timescales: a macro-timescale index k for long-term provisioning decisions, and a micro-timescale index t for real-time scheduling and system dynamics. Provisioning decisions remain fixed within each macro-period and are applied to all micro-timeslots within that period.

A. Network Architecture and Entity Definitions

The system operates over geographically distributed edge clusters. Let $C = \{1, 2, \dots, C\}$ denote the cluster-index set, where C is the number of clusters. For each cluster index $i \in C$, let $N_i = \{1, 2, \dots, N_i\}$ denote the set of compute-node indices in cluster i . To satisfy regional privacy and regulatory constraints, the global LoRA set $\mathcal{L}$ is partitioned into globally shared models $\mathcal{L}_{\text{Global}}$ and region-specific models $\mathcal{L}_{\text{Local}}^{(i)}$, such that

$$\mathcal{L} = \mathcal{L}_{\text{Global}} \cup \bigcup_{i \in C} \mathcal{L}_{\text{Local}}^{(i)} \quad (1)$$

with the sets being pairwise disjoint, i.e.,

$$\mathcal{L}_{\text{Global}} \cap \mathcal{L}_{\text{Local}}^{(i)} = \emptyset, \quad \forall i \in C, \quad (2)$$

$$\mathcal{L}_{\text{Local}}^{(i)} \cap \mathcal{L}_{\text{Local}}^{(i')} = \emptyset, \quad \forall i, i' \in C, i \neq i'. \quad (3)$$

Requests associated with region-specific LoRAs must remain within their originating clusters and are not eligible for cross-cluster offloading. Moreover, these region-specific LoRAs are mandatory at the macro-timescale provisioning layer and cannot be evicted from the persistent disk-level model store of their originating clusters, while transient VRAM or host-memory cache residency remains dynamically managed by the local scheduler. The system adopts a hierarchical three-tier architecture as follows [29]:

1) *Edge/Fog Orchestrator (EFO)*: The EFO serves as the global orchestration layer with a system-wide view. It performs long-term model provisioning by analyzing global workload patterns and determining the placement of shared LoRA models across clusters, thereby enabling macro-timescale storage planning. In addition, it maintains global resource and network states and periodically disseminates summarized system information—such as amortized computation costs and virtual queue backlogs—to support downstream decision-making.

2) *Control Node*: Each cluster is managed by a Control Node responsible for local scheduling and resource coordination. It executes real-time micro-scheduling decisions, including cross-cluster request routing and admission control under resource constraints. Furthermore, it manages cluster-level operations such as request dispatching, node power control (Wakeup/Standby), and VRAM-aware execution mode

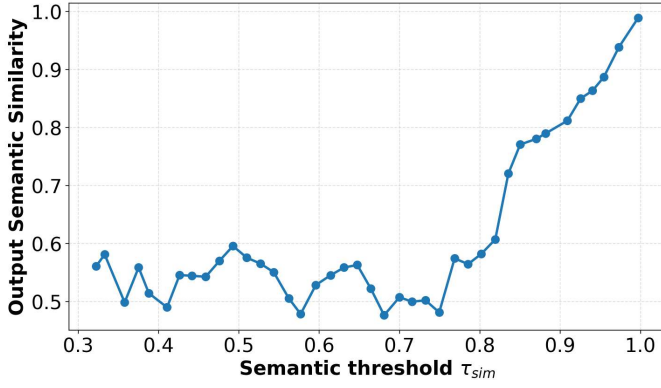


Fig. 2: Effect of semantic similarity threshold τ_{sim} on output semantic similarity.

switching (e.g., merged or unmerged LoRA execution) to balance performance and resource utilization.

3) *Compute Node*: The Compute Node is the execution unit within each cluster, equipped with a hierarchical memory architecture comprising host RAM and GPU VRAM. It executes scheduling decisions from the Control Node, maintains local model cache states, and performs inference using dynamic continuous batching to improve throughput and latency efficiency.

B. User Request and LoRA Semantic Model

In the edge federation, we define an inference request as a tuple $r = \langle \mathbf{d}_r, l_{req}(r), i_{src}(r), T_{max}^{(r)} \rangle$, where $\mathbf{d}_r$ denotes the input prompt, $l_{req}(r)$ is the target LoRA, $i_{src}(r) \in \mathcal{C}$ specifies the source cluster index for sovereignty enforcement, and $T_{max}^{(r)}$ represents the Time-To-First-Token (TTFT) constraint. In this work, we adopt a representative global latency target $\bar{T}_{max}$ to ensure QoE consistency and set $T_{max}^{(r)} = \bar{T}_{max}$ for all requests r (e.g., $\bar{T}_{max} = 6$ s in our evaluation). The request-level notation $T_{max}^{(r)}$ is retained for generality and can support heterogeneous SLO targets. In this work, the latency dimension of QoE is operationalized as TTFT-based SLO compliance. To standardize workloads under limited edge resources, we fix the input and output lengths to $D_{in} = 512$ and $D_{out} = 256$ tokens, respectively, where each token has a size of s_{tok} .

To improve cache efficiency and reduce cross-region transmission, we introduce a semantic substitution mechanism. Each LoRA l is associated with a semantic embedding vector $\mathbf{e}_l \in \mathbb{R}^d$, obtained from model-generated responses using a pre-trained encoder. The semantic similarity between two LoRA models is defined using cosine similarity:

$$\text{sim}(l_a, l_b) = \frac{\mathbf{e}_{l_a}^\top \mathbf{e}_{l_b}}{\|\mathbf{e}_{l_a}\|_2 \|\mathbf{e}_{l_b}\|_2}. \quad (4)$$

Empirical results (Fig. 2) indicate that high similarity correlates with comparable output semantic similarity in domain-specific tasks. When $\text{sim}(l_a, l_b) \geq \tau_{sim}$, the quality degradation remains negligible under standard evaluation metrics. Accordingly, we define the eligible substitute set as $\mathcal{E}(l_{req}(r)) = \{l \in \mathcal{L} \mid \text{sim}(l_{req}(r), l) \geq \tau_{sim}\}$, where τ_{sim} is a predefined

compatibility threshold. We set $\tau_{sim} = 0.9$ by default, as Fig. 2 shows that this value provides a practical operating point for maintaining high output semantic similarity while preserving sufficient substitution opportunities. We further define the reverse semantic coverage set $\mathcal{E}^{-1}(l) = \{l' \in \mathcal{L} \mid \text{sim}(l', l) \geq \tau_{sim}\}$ to characterize the substitution scope of each model. During inference, this mechanism enables the system to identify locally available substitutes, thereby reducing cold-start overhead.

C. VRAM-Constrained Execution Modes and Capacity Model

Compute nodes operate under two mutually exclusive VRAM execution modes with different computational overheads and batch capacities [30]. First, in the Merged Mode (Dedicated), LoRA weights are merged into the base model, incurring no additional computation overhead (i.e., $\theta_{mode} = 1$). Since VRAM is exclusively used for a single task type, the node achieves its merged-mode peak batch capacity, $\mathcal{N}_{batch}^{max}(i, j, t) = B_{max}^{merge}$. In contrast, the Unmerged Mode (Shared) enables multi-tenancy by dynamically loading multiple LoRA adapters, with peak batch capacity $B_{max}^{unmerge}$ before adapter-induced batch squeezing. This introduces additional computation overhead, modeled by $\theta_{mode} = 1 + \eta$, where $\eta \approx 0.26$ based on empirical profiling. Moreover, sharing VRAM among multiple adapters reduces the available space for KV cache, leading to the batch-squeezing effect.

The total VRAM capacity V_{total} must accommodate the base model, active adapters, and KV cache, which can be expressed as:

$$V_{total} \geq \underbrace{S_{base} + n_{lora}(\mathcal{B}_{i,j}(t)) \cdot S_{lora}}_{\text{Active Adapters}} + \underbrace{|\mathcal{B}_{i,j}(t)| \cdot S_{KV}}_{\text{KV Cache Usage}}. \quad (5)$$

Here, S_{base} denotes the base model size, S_{lora} the adapter footprint, and S_{KV} the KV cache size per request. We define B_{max}^{merge} and $B_{max}^{unmerge}$ as the peak batch capacities under the merged and unmerged execution modes after accounting for base model memory, respectively. Under the Unmerged Mode, increasing the number of active adapters n_{lora} reduces the available KV cache space, resulting in a reduced effective batch capacity:

$$\mathcal{N}_{batch}^{max}(i, j, t) = \begin{cases} B_{max}^{merge}, & m_{i,j}(t) \in \mathcal{L}, \\ \max \left\{ 0, B_{max}^{unmerge} - \left\lceil \frac{n_{lora}(\mathcal{B}_{i,j}(t)) S_{lora}}{S_{KV}} \right\rceil \right\}, & m_{i,j}(t) = 0. \end{cases} \quad (6)$$

D. End-to-End Latency Model and SLO Compliance

We model the Time-To-First-Token (TTFT) latency for each request. For a request r executed on node j in cluster i' , the end-to-end latency is defined as:

$$T_{TTFT}(r) = \Delta_{net}(r, i') + T_{wait}(r) + T_{load}(r, i', j) + T_{exec}(i', j, t). \quad (7)$$

The network delay Δ_{net} primarily accounts for the cross-cluster round-trip latency and prompt transmission delay. Since TTFT only involves returning the first generated token, whose payload size is negligible compared with the input prompt, we

TABLE II: List of Notations

Notation	Definition	Notation	Definition
k, t	Macro and micro time-slot indices	$C, \mathcal{N}_i$	Cluster-index set and compute nodes in cluster i
$\mathcal{L}_{\text{Global}}, \mathcal{L}_{\text{Local}}^{(i)}$	Shared and local LoRA sets	$S_i(k)$	LoRAs stored at cluster i in macro-period k
$\mathcal{R}_t, \mathcal{R}_{t,i}$	All requests and requests from cluster i at slot t	$\mathcal{B}_{i,j}(t)$	Batch assigned to node j in cluster i at slot t
$\mathcal{E}(l), \mathcal{E}^{-1}(l)$	Eligible substitutes and reverse coverage of LoRA l	$l_{\text{req}}(r), i_{\text{src}}(r)$	Requested LoRA and source cluster of request r
$\mathbf{e}_l, \tau_{\text{sim}}$	LoRA embedding and similarity threshold	$D_{\text{in}}, D_{\text{out}}, S_{\text{tok}}$	Input and output token lengths, and per-token size
$S_{\text{base}}, S_{\text{loras}}, S_{\text{KV}}$	VRAM sizes of base model, adapter, and KV cache	$n_{\text{loras}}(\mathcal{B}_{i,j}(t))$	Number of active LoRAs in a batch
$B_{\text{max}}^{\text{merge}}, B_{\text{max}}^{\text{unmerge}}, N_{\text{batch}}^{\text{max}}$	Peak batch sizes under merged/unmerged modes and dynamic batch limit	$B_i^{\text{compute}}, B_i^{\text{disk}}$	Node budget and disk capacity of cluster i
$\theta_{\text{mode}}, \eta$	Latency multiplier and unmerged-mode overhead	$\tau_{\text{pre}}, \alpha_{\text{int}}, \beta_{\text{new}}$	Prefill latency and contention factors
δ_{load}	Cold-start LoRA loading latency	$T_{\text{wait}}(r), T_{\text{load}}(r, i', j)$	Queueing and loading delays
$T_{\text{residency}}(r, i, j), T_{\text{step}}$	VRAM residency time and per-token latency	$\text{RTT}_{i,i'}, \text{BW}_{i,i'}(t)$	RTT and bandwidth between clusters
$\hat{T}_{\text{TFT}}(r), \hat{T}_{\text{max}}$	Estimated TTFT and SLO threshold	$\mathbf{Y} = \{y_{l,i}(k)\}$	Disk placement matrix
$\mathbf{N} = \{n_{i,j}(t)\}$	Node activation states	$\mathbf{M} = \{m_{i,j}(t)\}$	VRAM execution mode
$\mathbf{\Pi} = \langle \pi_{r,i'}, \mu_{r,i',j} \rangle$	Routing and dispatch decisions	$\mathbf{X} = \{x_{l,i,j}(t)\}$	VRAM LoRA residency states
$z_{r,i}(t), z_{r,i,j}^{\text{ram}}(t)$	Disk-feasible and VRAM-resident indicators	$\bar{z}_{l,i}(k)$	Macro-scale LoRA feasibility
$u_{i,j}(t), \rho_j$	GPU utilization and idle-power fraction	$J_{\text{inst}}, J_{\text{net,artifact}}, J_{\text{net,traffic}}, J_{\text{store}}$	Instance, network, and storage costs
J_{penalty}	Drop penalty cost	$\kappa_{i,j}^{\text{inst}}, \kappa_{i,j}^{\text{net}}, \kappa_{i,j}^{\text{store}}$	Unit compute, transfer, and storage costs
$\bar{\kappa}_i^{\text{inst}}, \kappa^{\text{WAN}}$	Cluster compute and WAN transfer costs	$\Psi_{\text{drop}}, \epsilon_{\text{slo}}$	Drop penalty and SLO tolerance
ϵ_{swap}	Swap trigger threshold in SP1	$\gamma_{i,i'}, \bar{T}_{\text{net},95}^{(i,i')}$	Urgency multiplier and 95th-percentile RTT
$\Delta\Lambda_i(l S_i)$	Marginal demand of adding LoRA l	$\lambda_{l,i}(\tau), \mathcal{W}_k$	Arrival count and history window
$\hat{\Lambda}_{l,i}(k)$	Predicted macro-period request count	$N_{i,j}^{\text{viol}}(t), A_i(t)$	SLO violations and served requests
$Z_i(t), V$	SLO-debt queue and control weight	$S_{i,j}^{\text{avail}}(t)$	Available KV-cache slots
$l^*, \mathcal{B}_m$	Candidate merged LoRA and request subset feasible under mode m	$\mathcal{Q}, \mathcal{B}_j^*$	Current request queue and tentative requests assigned to node j if activated
$\Phi_{\text{local},i,j}^r, \Phi_{\text{remote},i'}^r$	Local and remote Lyapunov dispatch costs	$c_{\text{dispatch}}^r(r, i \rightarrow i', j), \bar{c}_{\text{dispatch}}(l, i')$	Request-level and cluster-minimum amortized dispatch costs
$\Phi_{i,j}^r(m), \Phi_{\text{alt}}^r$	Mode-dependent dispatch cost and alternative penalty of request r	$\Phi_{i,j}^r$	Dispatch cost of assigning request r to node j in cluster i

omit its serialization/transmission time and absorb it into the RTT term:

$$\Delta_{\text{net}}(r, i') = \mathbb{I}(i_{\text{src}}(r) \neq i') \left(\text{RTT}_{i_{\text{src}}(r), i'} + \frac{D_{\text{in}} S_{\text{tok}}}{\text{BW}_{i_{\text{src}}(r), i'}(t)} \right), \quad (8)$$

where $\mathbb{I}(\cdot)$ indicates whether the request is offloaded, RTT is the round-trip time, and BW is the available bandwidth. The waiting time $T_{\text{wait}}(r)$ is estimated via time-window projection, while $T_{\text{load}}(r, i', j)$ denotes the model loading delay, incurring a cold-start penalty δ_{load} if no memory hit occurs.

The execution latency is modeled as:

$$T_{\text{exec}}(i', j, t) = \theta_{\text{mode}}(i', j, t) \left(\tau_{\text{pre}} + \alpha_{\text{int}} \cdot |\mathcal{B}_{i',j}(t)| + \beta_{\text{new}} \cdot n_{\text{new}}(i', j, t) \right), \quad (9)$$

where τ_{pre} is the base prefill time, α_{int} captures concurrency interference, and β_{new} models contention from newly arriving requests. Specifically, $n_{\text{new}}(i', j, t)$ denotes the number of newly admitted requests assigned to node j in cluster i' at time slot t that have not yet completed the prefill stage. The VRAM residency time is given by $T_{\text{residency}} = T_{\text{exec}} + D_{\text{out}} \cdot T_{\text{step}}$, where T_{step} denotes the average per-token decoding time after the first token. We enforce a probabilistic SLO such that at least 95% of served requests satisfy $T_{\text{TFT}}(r) \leq T_{\text{max}}^{(r)}$.

E. System Cost Model

We model the operational cost of the system in each time slot t as the aggregation of multiple physical expenditure components for each cluster $i \in C$. Provisioning-related costs

are determined by macro-timescale decisions and remain fixed within each macro-period. Specifically, four cost components are considered. Overall, the cost model is deployment-aware, traffic-aware, and service-aware.

1) *Energy-Proportional Instance Operating Cost*: The primary operational expenditure arises from GPU activation and utilization, following the energy-proportional computing model [31]. Based on the activation state $n_{i,j}(t)$ (indicating whether node j is active), utilization ratio $u_{i,j}(t)$, and idle cost factor ρ_j , the instance cost is defined as

$$J_{\text{inst}}^{(i)}(t) = \sum_{j \in \mathcal{N}_i} n_{i,j}(t) \cdot \kappa_{i,j}^{\text{inst}} \left(\rho_j + (1 - \rho_j) u_{i,j}(t) \right), \quad (10)$$

where $\kappa_{i,j}^{\text{inst}}$ denotes the unit compute cost of node j , and ρ_j captures the fraction of cost incurred under idle operation. The utilization ratio is given by

$$u_{i,j}(t) = \frac{1}{\Delta t} \int_{t' \in t} \mathbb{I}(|\mathcal{B}_{i,j}(t')| > 0) dt', \quad (11)$$

representing the fraction of time during which node j actively serves requests within time slot t .

2) *Network Transmission Cost*: This component includes both model provisioning and inference traffic. The artifact download cost is

$$J_{\text{net,artifact}}^{(i)}(t) = \sum_{l \in \mathcal{L}} \left[y_{l,i}(k) - y_{l,i}(k-1) \right]^+ S_{\text{loras}} \cdot \kappa^{\text{WAN}}, \quad (12)$$

where $y_{l,i}(k) \in \{0, 1\}$ indicates whether LoRA l is stored in cluster i , and $[\cdot]^+$ denotes the positive part operator.

The inference traffic cost is defined as

$$J_{\text{net,traffic}}^{(i)}(t) = s_{\text{tok}} \sum_{r \in \mathcal{R}_{t,i}} (D_{\text{in}} + D_{\text{out}}) \left(\pi_{r,i} \cdot \kappa_{i,i}^{\text{net}} + \sum_{i' \in C \setminus \{i\}} \pi_{r,i'} \cdot \kappa_{i,i'}^{\text{net}} \right), \quad (13)$$

where $\mathcal{R}_{t,i}$ is the set of requests originating from cluster i at time t , $\pi_{r,i'} \in \{0, 1\}$ denotes the routing decision of request r , and $\kappa_{i,i'}^{\text{net}}$ represents the unit transmission cost between clusters i and i' .

3) *Storage Retention Cost*: The storage cost of locally retained LoRA is

$$J_{\text{store}}^{(i)}(t) = \sum_{l \in \mathcal{L}} y_{l,i}(k) S_{\text{loRa}} \cdot \kappa^{\text{store}}, \quad (14)$$

where κ^{store} is the unit storage cost.

4) *Rejection Penalty*: To discourage excessive request dropping, we impose a penalty:

$$J_{\text{penalty}}^{(i)}(t) = \sum_{r \in \mathcal{R}_{t,i}} \pi_{r,0} \cdot \Psi_{\text{drop}}, \quad (15)$$

where $\pi_{r,0} = 1$ indicates that request r is rejected, and Ψ_{drop} is a penalty coefficient.

Finally, the total cost for cluster i is

$$J^{(i)}(t) = J_{\text{inst}}^{(i)}(t) + J_{\text{net,artifact}}^{(i)}(t) + J_{\text{net,traffic}}^{(i)}(t) + J_{\text{store}}^{(i)}(t) + J_{\text{penalty}}^{(i)}(t). \quad (16)$$

IV. PROBLEM FORMULATION

In this section, we formulate the optimization objective of the system, which is to minimize the total operational cost of the multi-cluster edge system while satisfying strict Quality of Experience (QoE) constraints. Due to the inherent heterogeneity in decision timescales and system operations between model provisioning and real-time inference dispatching, we decouple the original joint optimization problem into two sub-problems. Specifically, Sub-problem 1 (SP1) is defined at the macro-timescale period (k) for global LoRA model provisioning, while Sub-problem 2 (SP2) is defined at the micro-timescale slot (t) for real-time request routing and node control decisions.

A. Global Optimization Objective

The global optimization objective is to minimize the cumulative operational cost of all edge clusters over the total operational period T_{total} . To satisfy physical resource limits and QoE requirements, the objective is subject to a set of system constraints. It is formally defined as follows:

$$\min_{\mathbf{Y}, \mathbf{N}, \mathbf{M}, \mathbf{\Pi}} \mathcal{J}_{\text{Global}} = \sum_{t=1}^{T_{\text{total}}} \sum_{i \in C} \left(J_{\text{inst}}^{(i)}(t) + J_{\text{net,artifact}}^{(i)}(t) + J_{\text{net,traffic}}^{(i)}(t) + J_{\text{store}}^{(i)}(t) + J_{\text{penalty}}^{(i)}(t) \right). \quad (17)$$

s.t.

$$\mathbf{C1}: \pi_{r,0} + \sum_{i' \in C} \pi_{r,i'} = 1, \quad \forall r \in \mathcal{R}_t,$$

$$\begin{aligned} & \sum_{j \in \mathcal{N}_{i'}} \mu_{r,i',j} = \pi_{r,i'}, \quad \forall r \in \mathcal{R}_t, \forall i' \in C, \\ \mathbf{C2}: & \sum_{j \in \mathcal{N}_i} n_{i,j}(t) \leq B_i^{\text{compute}}, \quad \forall i \in C, \\ & \sum_{l \in \mathcal{L}} y_{l,i}(k) S_{\text{loRa}} \leq B_i^{\text{disk}}, \quad \forall i \in C, \\ & |\mathcal{B}_{i,j}(t)| \leq \mathcal{N}_{\text{batch}}^{\text{max}}(i, j, t), \quad \forall i \in C, \forall j \in \mathcal{N}_i, \forall t, \\ \mathbf{C3}: & m_{i,j}(t) = l, \quad l \in \mathcal{L} \\ & \implies l \in \mathcal{E}(l_{\text{req}}(r)), \quad \forall r \in \mathcal{B}_{i,j}(t), \\ & \frac{\sum_{\tau=1}^t \sum_{r \in \mathcal{R}_{\tau}} (1 - \pi_{r,0}(\tau)) \mathbb{I}(\hat{T}_{\text{TTFT}}(r, \tau) \leq T_{\text{max}}^{(r)})}{\max\{1, \sum_{\tau=1}^t \sum_{r \in \mathcal{R}_{\tau}} (1 - \pi_{r,0}(\tau))\}} \geq 0.95, \\ & \pi_{r,i'} \leq z_{r,i'}(t), \quad \forall r \in \mathcal{R}_t, \forall i' \in C, \\ & l_{\text{req}}(r) \in \mathcal{L}_{\text{Local}}^{(i_{\text{src}}(r))} \implies \sum_{i' \in C \setminus \{i_{\text{src}}(r)\}} \pi_{r,i'} = 0, \\ & y_{l,i}(k) = 1, \quad \forall l \in \mathcal{L}_{\text{Local}}^{(i)}, \forall k, \\ \mathbf{C5}: & \mu_{r,i',j} \leq n_{i',j}(t), \quad \forall r \in \mathcal{R}_t, \forall i' \in C, \forall j \in \mathcal{N}_{i'}. \end{aligned} \quad (18)$$

For notational compactness, macro-period provisioning and storage costs are aggregated over the same operational horizon as micro-slot dispatching costs. However, $J_{\text{net,artifact}}^{(i)}(t)$ and $J_{\text{store}}^{(i)}(t)$ are updated only when the provisioning decision $y_{l,i}(k)$ changes at macro-period boundaries.

To characterize the decision space of the three-tier collaborative architecture, we define four groups of active decision variables: $\mathbf{Y}$ (provisioning), $\mathbf{N}$ (node activation), $\mathbf{M}$ (execution mode), and $\mathbf{\Pi} = \langle \boldsymbol{\pi}, \boldsymbol{\mu} \rangle$ (routing). Specifically, $y_{l,i}(k) \in \{0, 1\}$ indicates whether LoRA l is provisioned at cluster i during macro-period k , and $n_{i,j}(t) \in \{0, 1\}$ indicates whether node j in cluster i is active at time slot t . The execution mode variable $m_{i,j}(t) \in \{0\} \cup \mathcal{L}$ specifies the VRAM mode of node (i, j) at time slot t , where $m_{i,j}(t) = 0$ denotes the shared (unmerged) mode and $m_{i,j}(t) \in \mathcal{L}$ denotes the dedicated (merged) mode. The routing variable $\mathbf{\Pi}$ comprises cluster-level decisions $\boldsymbol{\pi}$ (forwarding and dropping) and node-level dispatching $\boldsymbol{\mu}$. Formal definitions are summarized in Table II.

In addition, we introduce a passive state variable $x_{l,i,j}(t) \in \{0, 1\}$ to capture VRAM residency, indicating whether LoRA l is cached on node j of cluster i . Based on $\mathbf{Y}$ and $\mathbf{X}$, we define the following derived feasibility indicators:

$$z_{r,i}(t) = \max_{l \in \mathcal{E}(l_{\text{req}}(r))} y_{l,i}(k), \quad z_{r,i,j}^{\text{ram}}(t) = \max_{l \in \mathcal{E}(l_{\text{req}}(r))} x_{l,i,j}(t),$$

which capture disk-level semantic feasibility at destination cluster i and cache-resident substitutability at node (i, j) , respectively.

The global optimization is subject to five constraints: **C1** enforces unique routing; **C2** imposes compute, storage, and VRAM capacity limits; **C3** ensures that a merged-mode node serves only requests semantically compatible with its merged LoRA; **C4** ensures TTFT SLO, semantic feasibility, and data sovereignty; and **C5** guarantees that requests are dispatched only to active nodes.

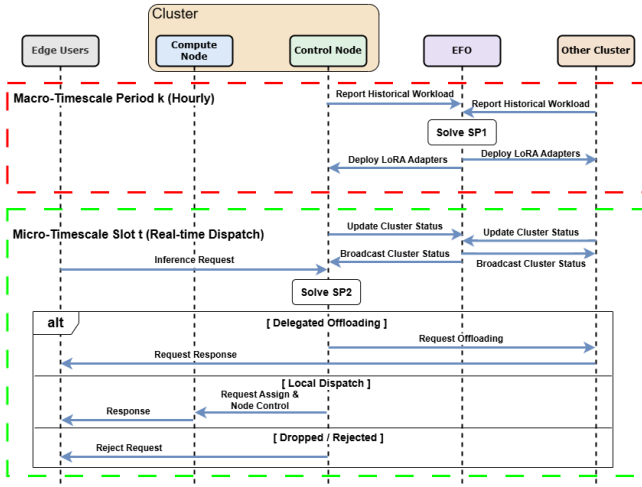


Fig. 3: Overview of hierarchical multi-timescale inference system workflow.

B. Hierarchical Multi-timescale Problem Decomposition

Directly solving the global joint optimization problem is computationally intractable due to the mismatch in decision timescales and state update frequencies between long-term provisioning and real-time dispatching. Therefore, as shown in Fig. 3, we decompose the problem into two hierarchical sub-problems within the system control architecture. At the macro-timescale (e.g., hourly), the EFO performs the Provisioning Phase (SP1), utilizing predicted semantic demand to determine the global placement matrix $\mathbf{Y}$, which defines local service boundaries. Given these provisioning decisions, at the micro-timescale (millisecond-level), each cluster's Control Node executes the Dispatching Phase (SP2), dynamically optimizing request routing $\mathbf{\Pi}$, node activation $\mathbf{N}$, and VRAM execution modes $\mathbf{M}$ based on real-time system states and EFO-provided summaries.

1) *Sub-problem 1 (SP1): Cost-Aware Predictive LoRA Placement:* We formulate sub-problem **P1**, solved by the Edge/Fog Orchestrator (EFO) at the macro-timescale provisioning phase. Due to uncertainty in future arrivals, request traffic is approximated by the predicted aggregate count $\hat{\Lambda}_{l,i}(k)$ from LSTM forecasting. Since future network states and micro-level decisions are unknown, operational costs and latency penalties are replaced by expected values. Incorporating deployment and storage costs, **P1** is defined as:

$$\begin{aligned} \mathbf{P1} : \min_{\mathbf{Y}} \quad & \mathcal{J}_{SP1}(k) = J_{\text{deploy}}(k) + J_{\text{store}}(k) + J_{\text{ops}}^{\text{expected}}(k) \\ \text{s.t.} \quad & \mathbf{C2}_{\text{storage}}, \mathbf{C4}_{\text{macro}}. \end{aligned} \quad (19)$$

Here, $\mathbf{C2}_{\text{storage}}$ denotes the macro-timescale storage-capacity constraint, requiring that the total size of LoRA adapters provisioned at each cluster does not exceed its disk budget. $\mathbf{C4}_{\text{macro}}$ represents the macro-level feasibility constraint, which enforces data-sovereignty requirements by mandating the placement of region-specific LoRAs in their originating clusters and restricting them from being served through cross-region provisioning.

The first two terms aggregate physical costs across clusters. The deployment cost $J_{\text{deploy}}(k) = \sum_{i \in \mathcal{C}} J_{\text{net,artifact}}^{(i)}(k)$ captures network overhead for provisioning LoRA models, while the storage cost $J_{\text{store}}(k) = \sum_{i \in \mathcal{C}} J_{\text{store}}^{(i)}(k)$ accounts for local retention.

The expected operational cost integrates computation and transmission under predicted demand. To account for semantic substitution, we replace the exact-placement indicator $y_{l,i}(k)$ with the macro-timescale semantic feasibility indicator $\bar{z}_{l,i}(k) \triangleq \max_{l' \in \mathcal{E}(l)} y_{l',i}(k)$, which is the aggregate counterpart of $z_{r,i}(t)$:

$$\begin{aligned} J_{\text{ops}}^{\text{expected}}(k) = \sum_{i \in \mathcal{C}} \sum_{l \in \mathcal{L}} \hat{\Lambda}_{l,i}(k) & \left(\bar{z}_{l,i}(k) \bar{\kappa}_i^{\text{inst}} + (1 - \bar{z}_{l,i}(k)) \right. \\ & \left. \min \left\{ \Psi_{\text{drop}}, \min_{i' \neq i: \bar{z}_{l,i'}(k)=1} (\gamma_{i,i'} \bar{\kappa}_{i'}^{\text{inst}} + s_{\text{tok}}(D_{\text{in}} + D_{\text{out}}) \kappa_{i,i'}^{\text{net}}) \right\} \right). \end{aligned} \quad (20)$$

The first term represents local execution when the requested LoRA is semantically feasible in the local cluster, while the second captures the minimum expected cost of remote execution when no local feasible substitute is available, with the remote minimum evaluated as $+\infty$ if no feasible remote cluster exists. In this case, the request is handled through the drop option and incurs the corresponding drop penalty.

To model latency risk in cross-region offloading, we introduce an ‘‘Urgency Multiplier’’ $\gamma_{i,i'}$ based on the 95th percentile RTT $\bar{T}_{\text{net},95}^{(i,i')}$, reflecting tail latency instead of average delay. It converts the gap to the TTFT threshold $\bar{T}_{\text{max}}$ into a penalty:

$$\gamma_{i,i'} = \begin{cases} 1, & \text{if } i = i' \\ \frac{\bar{T}_{\text{max}}}{\bar{T}_{\text{max}} - \bar{T}_{\text{net},95}^{(i,i')}}, & \text{if } 0 < \bar{T}_{\text{net},95}^{(i,i')} < \bar{T}_{\text{max}} \\ +\infty, & \text{if } \bar{T}_{\text{net},95}^{(i,i')} \geq \bar{T}_{\text{max}} \end{cases} \quad (21)$$

As cross-region delay approaches the latency limit, the penalty grows sharply, discouraging offloading and promoting local deployment for latency-sensitive models. The objective minimizes the total expected cost during macro-period k . Since $\mathbf{Y}$ is binary, **P1** reduces to a Dynamic Multiple Knapsack Problem, which is NP-hard.

2) *Sub-problem 2 (SP2): Real-Time Dispatching and Execution:* We define sub-problem **P2**, solved in a distributed manner by cluster Control Nodes during the real-time dispatching phase. Unlike macro-timescale provisioning, inference routing and VRAM mode switching operate at millisecond granularity. The objective is to minimize instantaneous operational cost within a micro-slot t :

$$\begin{aligned} \mathbf{P2} : \min_{\mathbf{N}, \mathbf{M}, \mathbf{\Pi}} \quad & \mathcal{J}_{SP2}^{\text{Global}}(t) = \sum_{i \in \mathcal{C}} \left(J_{\text{inst}}^{(i)}(t) + J_{\text{net,traffic}}^{(i)}(t) \right. \\ & \left. + J_{\text{penalty}}^{(i)}(t) \right) \\ \text{s.t.} \quad & \mathbf{C1}, \mathbf{C2}, \mathbf{C3}, \mathbf{C4}, \mathbf{C5} \end{aligned} \quad (22)$$

Since **P2** is evaluated at slot t , all cost terms directly follow the system cost model (Section III-E). Specifically, the scheduler balances GPU computation cost $J_{\text{inst}}^{(i)}(t)$ (driven by node activation $\mathbf{N}$), traffic cost $J_{\text{net,traffic}}^{(i)}(t)$ (from routing $\mathbf{\Pi}$), and penalty cost $J_{\text{penalty}}^{(i)}(t)$ (from request dropping). The

decision variables jointly determine node activation, VRAM modes $\mathbf{M}$, and routing within the slot.

Although **P1** and **P2** are not strictly equivalent to $\mathcal{J}_{\text{Global}}$, the decomposition enables tractable optimization with incomplete future information. To reduce suboptimality and enforce long-term constraints in real time, Section V adopts Lyapunov optimization, transforming SLO violation probability into a virtual queue and converting the problem into per-slot control, thereby approximating the long-term cost-QoE objective.

V. PROPOSED METHOD

To approximate the global objective $\mathcal{J}_{\text{Global}}$, we develop a cross-timescale collaborative framework. It comprises a two-phase semantic-aware provisioning algorithm for macro-timescale SP1 and a Lyapunov-based dispatching algorithm for micro-timescale SP2, which jointly optimize routing and resource allocation under QoE constraints, thereby approximating the global objective while minimizing operational cost. Fig. 3 illustrates how SP1 defines the feasible service boundary under provisioning and sovereignty constraints, while SP2 performs real-time control within that boundary to adapt to workload dynamics.

A. Solving SP1: Cost-Aware Predictive LoRA Provisioning

SP1 is formulated as a Dynamic Multiple Knapsack Problem with semantic substitution, incorporating constraints, transition costs, and semantic correlations. At each macro-period k , the EFO applies a two-phase provisioning algorithm based on global workload forecasts to determine the placement matrix $\mathbf{Y}$, which guides micro-timescale dispatching.

1) *Macro-Timescale Workload Forecasting Model*: To enable accurate provisioning, the system predicts the aggregated request count $\hat{\Lambda}_{l,i}(k)$ for each model l at cluster i . To address cold-start and demand surges, we adopt an LSTM with multi-dimensional feature fusion:

$$\hat{\Lambda}_{l,i}(k) = \exp\left(\text{LSTM}(\mathbf{w}_{l,i}(k-1), \mathbf{h}(k-1), \mathbf{e}_l; \theta_{\text{LSTM}})\right) - 1 \quad (23)$$

The input includes: (i) log-smoothed historical traffic $\mathbf{w}_{l,i}(k) = \ln(1 + \sum_{\tau \in \mathcal{W}_k} \lambda_{l,i}(\tau))$ to mitigate burstiness, where $\lambda_{l,i}(\tau)$ denotes the observed arrival count of LoRA l at cluster i during historical micro-slot τ and $\mathcal{W}_k$ is the historical observation window for macro-period k ; (ii) temporal feature $\mathbf{h}(k)$, such as hour-of-day, day-of-week, or periodic encoding, for capturing workload periodicity; and (iii) semantic embedding $\mathbf{e}_l$. The parameter θ_{LSTM} denotes the trainable parameters of the forecasting model.

2) *Semantic Coverage and Marginal Demand Definition*: Under semantic substitution, a model can serve requests beyond itself. To quantify deployment benefits without double-counting, we define coverage relationships. Let $\mathcal{S}_i(k)$ denote the set of models stored at cluster i during macro-period k . The demand space already covered by these deployed models is defined as $\mathcal{V}_{\text{cov}}(\mathcal{S}_i(k)) = \bigcup_{l \in \mathcal{S}_i(k)} \mathcal{E}^{-1}(l)$. Based on this covered demand space, the Semantic Marginal Demand of adding a candidate model l_{cand} is:

$$\Delta\Lambda_i(l_{\text{cand}} | \mathcal{S}_i(k)) = \sum_{l' \in \mathcal{E}^{-1}(l_{\text{cand}}) \setminus \mathcal{V}_{\text{cov}}(\mathcal{S}_i(k))} \hat{\Lambda}_{l',i}(k) \quad (24)$$

Algorithm 1: Two-Phase Semantic-Aware Provisioning Algorithm

Input: Predicted demand $\hat{\Lambda}_{l,i}(k)$, marginal demand $\Delta\Lambda_i$, previous placement $\mathcal{S}_i(k-1)$, disk capacities B_i^{disk}

Output: Placement matrix $\mathbf{Y}(k)$

- 1 Initialize $\mathbf{Y}(k)$ and $\mathcal{S}_i(k) \leftarrow \mathcal{S}_i(k-1)$ for all i ;
- 2 **foreach** cluster index $i \in C$ // Phase 1
- 3 Retain all mandatory regional models in cluster i ;
- 4 **while** *true* **do**
- 5 $l_{\text{cand}} \leftarrow \arg \max_{l \in \mathcal{L}_{\text{Global}} \setminus \mathcal{S}_i(k)} U_{\text{expand}}^{\text{local}}(l, i)$;
- 6 **if** $U_{\text{expand}}^{\text{local}}(l_{\text{cand}}, i) \leq \epsilon_{\text{swap}}$ **then**
- 7 **break**
- 8 **if** *cluster i has sufficient remaining capacity* **then**
- 9 Add l_{cand} to $\mathcal{S}_i(k)$;
- 10 **else**
- 11 $l_{\text{min}} \leftarrow \arg \min_{v \in \mathcal{S}_i(k) \setminus \mathcal{L}_{\text{Local}}^{(i)}} U_{\text{loss}}^{\text{local}}(v, i)$;
- 12 **if** $U_{\text{expand}}^{\text{local}}(l_{\text{cand}}, i) > U_{\text{loss}}^{\text{local}}(l_{\text{min}}, i) + \epsilon_{\text{swap}}$ **then**
- 13 Swap l_{min} with l_{cand} ;
- 14 **else**
- 15 **break**
- 16 Let $\text{NetGain}(l, i)$ be $U_{\text{expand}}^{\text{global}}(l, i)$ for direct placement, minus $U_{\text{loss}}^{\text{global}}(l_{\text{min}}, i)$ only if replacement is needed.
- 17 **foreach** $l_{\text{global}} \in \{l \in \mathcal{L}_{\text{Global}} \mid \max_{i \in C: l \notin \mathcal{S}_i(k)} \text{NetGain}(l, i) > \epsilon_{\text{swap}}\}$ // Phase 2
- 18 Find $i^* = \arg \max_{i \in C: l_{\text{global}} \notin \mathcal{S}_i(k)} \text{NetGain}(l_{\text{global}}, i)$;
- 19 **if** $\text{NetGain}(l_{\text{global}}, i^*) > \epsilon_{\text{swap}}$ **then**
- 20 Place l_{global} at i^* by direct addition or by replacing the minimum-loss removable model;
- 21 **return** $\mathbf{Y}(k)$;

This definition ensures that only newly covered predicted traffic is counted, providing a consistent basis for subsequent greedy selection.

3) *Two-Phase Semantic-Aware Provisioning*: After obtaining predicted workloads and marginal demands, provisioning is performed in two phases to balance regional autonomy and global efficiency (Algorithm 1).

Phase 1: Regional Deployment. Each cluster $i \in C$ independently performs greedy provisioning based on local demand. It first enforces data sovereignty by retaining all regional models ($y_{l,i}(k) = 1, \forall i \in C, \forall l \in \mathcal{L}_{\text{Local}}^{(i)}$). The remaining models are ranked by expected expansion utility $U_{\text{expand}}^{\text{local}}$ and selected iteratively. The expansion benefit and removal loss are:

$$U_{\text{expand}}^{\text{local}}(l_{\text{cand}}, i) = \Delta\Lambda_i(l_{\text{cand}} | \mathcal{S}_i) \cdot \Psi_{\text{drop}} - S_{\text{lora}}(\kappa^{\text{store}} + \kappa^{\text{WAN}}) \quad (25)$$

$$U_{\text{loss}}^{\text{local}}(l_{\text{min}}, i) = \Delta\Lambda_i(l_{\text{min}} | \mathcal{S}_i \setminus \{l_{\text{min}}\}) \cdot \Psi_{\text{drop}} \quad (26)$$

Here, $U_{\text{expand}}^{\text{local}}$ captures penalty reduction minus deployment cost, while $U_{\text{loss}}^{\text{local}}$ quantifies service loss. Under capacity limits, the model with the lowest marginal contribution is replaced. To avoid oscillation, a swap is performed only if $U_{\text{expand}}^{\text{local}} > U_{\text{loss}}^{\text{local}} + \epsilon_{\text{swap}}$. The process continues until all gains fall below ϵ_{swap} .

Phase 2: Global Semantic Compensation. To address under-provisioned long-tail models, the EFO performs a global adjustment. The global expansion gain and removal loss are:

$$U_{\text{expand}}^{\text{global}}(l_{\text{global}}, i) = \sum_{q \in C} \Delta \Lambda_q(l_{\text{global}} | S_q) \cdot \max\left(0, \Psi_{\text{drop}} - (\gamma_{q,i} \bar{\kappa}_i^{\text{inst}} + \kappa_{q,i}^{\text{net}}(D_{\text{in}} + D_{\text{out}})s_{\text{tok}})\right) - S_{\text{loa}}(\kappa^{\text{store}} + \kappa^{\text{WAN}}) \quad (27)$$

$$U_{\text{loss}}^{\text{global}}(l_{\text{min}}, i) = \sum_{q \in C} \Delta \Lambda_q(l_{\text{min}} | S_q \setminus \{l_{\text{min}}\}) \cdot \max\left(0, \Psi_{\text{drop}} - (\gamma_{q,i} \bar{\kappa}_i^{\text{inst}} + \kappa_{q,i}^{\text{net}}(D_{\text{in}} + D_{\text{out}})s_{\text{tok}})\right) \quad (28)$$

Here, $\bar{\kappa}_i^{\text{inst}}$ and $\kappa_{q,i}^{\text{net}}$ denote computation and transmission costs, while $\gamma_{q,i}$ accounts for latency risk. Let l_{min} denote the currently placed model with the minimum replacement priority at cluster i when replacement is required; otherwise, the corresponding removal loss is set to zero. The optimal placement is selected as:

$$i^* = \arg \max_{i \in C} (U_{\text{expand}}^{\text{global}}(l_{\text{global}}, i) - U_{\text{loss}}^{\text{global}}(l_{\text{min}}, i)) \quad (29)$$

The resulting placement matrix $\mathbf{Y}(k)$ provides the foundation for determining the real-time feasibility indicator $z_{r,i}(t)$ in SP2.

B. Solving SP2: Real-Time Dispatching via Lyapunov Optimization

Given the macro-level decisions $\mathbf{Y}(k)$ from SP1, the system performs real-time control at micro-slot t . Using Lyapunov optimization, it jointly determines routing $\mathbf{\Pi}$, execution mode $\mathbf{M}$, and node activation $\mathbf{N}$ under dynamic VRAM constraints. This framework balances instantaneous cost and long-term latency compliance without requiring future information. The control parameter V trades off cost and QoE, achieving an $O(1/V)$ optimality gap with an $O(V)$ delay.

To enforce the long-term 95% TTFT-SLO compliance constraint, we introduce a virtual queue $Z_i(t)$ for each cluster:

$$Z_i(t+1) = \max\left[0, Z_i(t) + N_i^{\text{viol}}(t) - \epsilon_{\text{slo}} \cdot A_i(t)\right] \quad (30)$$

where $N_i^{\text{viol}}(t)$ is the number of violations, $A_i(t)$ the total served requests, and $\epsilon_{\text{slo}} = 0.05$. The queue $Z_i(t)$ represents accumulated latency debt. Using the Drift-Plus-Penalty (DPP)

framework, SP2 reduces to minimizing the per-slot bound $\mathcal{L}_{\text{SP2}}^{(i)}(t)$:

$$\min_{\mathbf{N}, \mathbf{M}, \mathbf{\Pi}} \sum_{i \in C} \left\{ V \cdot \left(J_{\text{inst}}^{(i)}(t) + J_{\text{net,traffic}}^{(i)}(t) + J_{\text{penalty}}^{(i)}(t) \right) + \sum_{r \in \mathcal{R}_{i,i}} \left[Z_i(t) \sum_{j \in \mathcal{N}_i} \mu_{r,i,j} \left(\mathbb{I}(\hat{T}_{\text{TTFT}}^{\text{local}} > T_{\text{max}}^{(r)} - \epsilon_{\text{slo}}) \right) + \sum_{i' \neq i} Z_{i'}(t) \pi_{r,i'} \left(\mathbb{I}(\hat{T}_{\text{TTFT}}^{\text{remote}} > T_{\text{max}}^{(r)} - \epsilon_{\text{slo}}) \right) \right] \right\} \quad (31)$$

where $\hat{T}_{\text{TTFT}}^{\text{local}}$ and $\hat{T}_{\text{TTFT}}^{\text{remote}}$ denote expected latency for local and remote execution.

To avoid VRAM over-allocation, each node tracks KV-cache usage. The available slot budget of node j is:

$$S_{i,j}^{\text{avail}}(t) = \mathcal{N}_{\text{batch}}^{\text{max}}(i, j, t) - |\mathcal{R}_{\text{prev}}(i, j, t)| \quad (32)$$

where $\mathcal{N}_{\text{batch}}^{\text{max}}$ is the maximum batch size and $\mathcal{R}_{\text{prev}}(i, j, t)$ denotes the set of residual requests that have already been assigned to node j in cluster i before the current routing decision at time slot t . This captures the batch reduction caused by multiple LoRA loading, i.e., the batch-squeezing effect, and requires $S_{i,j}^{\text{avail}}(t) \geq 1$ for feasible routing.

The following subsections detail the real-time decision process for minimizing the DPP objective under these constraints (Algorithm 2). For remote offloading, Algorithm 2 determines only the cluster-level routing decision. The node-level dispatching variable μ is subsequently determined by the destination cluster's Control Node.

1) Look-Ahead Routing and Delegated Offloading: During routing, the Control Node estimates future queueing delays via time-window projection and combines operational cost with SLO penalties into a unified dispatch cost. The amortized compute cost for dispatching request r from cluster i to node j in cluster i' is:

$$c_{\text{dispatch}}(r, i \rightarrow i', j) = \begin{cases} \infty, & \text{if } z_{r,i'}(t) = 0 \\ \kappa_{i',j}^{\text{inst}} \frac{T_{\text{load}}(r, i', j) + T_{\text{residency}}(r, i', j)}{\mathcal{N}_{\text{batch}}^{\text{max}}(i', j, t; \mathcal{B}_{i',j} \cup \{r\})}, & \text{if } z_{r,i'}(t) = 1 \end{cases} \quad (33)$$

where $T_{\text{load}}(r, i', j)$ is the cold-start latency and $T_{\text{residency}}(r, i', j)$ the VRAM residency time. The feasibility indicator $z(t)$ filters invalid routes: disk misses ($z_{r,i'}(t) = 0$) yield infinite cost, while memory hits eliminate loading cost. To capture cross-region effects, the EFO periodically broadcasts global states (e.g., $\bar{c}_{\text{dispatch}}, Z_{i'}$), where $\bar{c}_{\text{dispatch}}(l, i')$ denotes the minimum dispatch cost for serving LoRA demand l in cluster i' , i.e., $\bar{c}_{\text{dispatch}}(l, i') = \min_{j \in \mathcal{N}_{i'}} c_{\text{dispatch}}(r, i' \rightarrow i', j)$ for requests with $l_{\text{req}}(r) = l$. This enables cost decomposition. In the tentative assignment test, $\mathcal{N}_{\text{batch}}^{\text{max}}(i', j, t; \mathcal{B}_{i',j} \cup \{r\})$ denotes the effective batch capacity recomputed after adding request r to the current batch, including the resulting number of active LoRA adapters and the corresponding KV-cache availability.

The system evaluates local and remote execution costs, denoted by $\Phi_{\text{local},i,j}^r$ and $\Phi_{\text{remote},i',j}^r$, respectively. Here, $\Phi_{\text{local},i,j}^r$

Algorithm 2: Lyapunov-Driven Real-Time Dispatching Algorithm

Input: Requests $\mathcal{R}_{t,i}$, placement $\mathbf{Y}(k)$, virtual queue $Z_i(t)$, EFO states $\{\bar{c}_{\text{dispatch}}, Z_{i'}\}$

Output: Macro-routing π , micro-dispatching μ , execution modes $\mathbf{M}(t)$, node states $\mathbf{N}(t)$

```

1 Initialize  $\pi \leftarrow 0, \mu \leftarrow 0$ ;
2 foreach request  $r \in \mathcal{R}_{t,i}$  // Routing
3   Find the best local action  $(j^*, \Phi_{\text{local}}^{r,*})$ ;
4   Find the best remote action  $(i_{\text{remote}}^{r,*}, \Phi_{\text{remote}}^{r,*})$ ;
5   Select the minimum-cost action;
6   if  $\Phi_{\text{local}}^{r,*} \leq \min(\Phi_{\text{remote}}^{r,*}, V\Psi_{\text{drop}})$  then
7      $\pi_{r,i} \leftarrow 1, \mu_{r,i,j^*} \leftarrow 1$ ;
8     Update  $S_{i,j^*}^{\text{avail}}(t)$ ;
9   else if  $\Phi_{\text{remote}}^{r,*} \leq V\Psi_{\text{drop}}$  then
10     $\pi_{r,i_{\text{remote}}^{r,*}} \leftarrow 1$ ;
11  else
12     $\pi_{r,0} \leftarrow 1$ ;
13 foreach active node  $j \in \mathcal{N}_i$  // Mode Selection
14   Select  $m_{i,j}(t) =$ 
15      $\arg \min_{m \in \{0, l^*\}} \left( \sum_{r \in \mathcal{B}_m} \Phi_{i,j}^r(m) + \sum_{r \in Q \setminus \mathcal{B}_m} \Phi_{\text{alt}}^r \right)$ ;
16 foreach node  $j \in \mathcal{N}_i$  // Auto-Scaling
17   Compute drift benefit  $\Delta\Phi_{\text{net}}(j)$ ;
18   if  $\Delta\Phi_{\text{net}}(j) > 0$  and budget allows then
19      $n_{i,j}(t) \leftarrow 1$ ;
20   else if  $j$  is idle and  $\Delta\Phi_{\text{net}}(j) \leq 0$  then
21      $n_{i,j}(t) \leftarrow 0$ ;
22 Let  $A_i(t)$  and  $N_i^{\text{viol}}(t)$  be computed over served requests with  $\pi_{r,0} = 0$ ;
23 Update
     $Z_i(t+1) \leftarrow \max[0, Z_i(t) + N_i^{\text{viol}}(t) - \epsilon_{\text{slo}} A_i(t)]$ ;
24 return  $\pi, \mu, \mathbf{M}(t), \mathbf{N}(t)$ ;

```

represents the Lyapunov drift-plus-penalty cost of serving request r locally at node j in source cluster i , while $\Phi_{\text{remote},i'}^r$ represents the corresponding cost of offloading request r to remote cluster i' .

$$\Phi_{\text{local},i,j}^r = V \cdot c_{\text{dispatch}}(r, i \rightarrow j) + Z_i(t) [\mathbb{I}(\hat{T}_{\text{TTFT}} > T_{\text{max}}^{(r)}) - \epsilon_{\text{slo}}] \quad (34)$$

$$\Phi_{\text{remote},i'}^r = V \cdot (\bar{c}_{\text{dispatch}}(l_{\text{req}}(r), i') + \kappa_{i,i'}^{\text{net}}(D_{\text{in}} + D_{\text{out}})s_{\text{tok}}) + Z_{i'}(t) [\mathbb{I}(\hat{T}_{\text{TTFT}}^{\text{remote}} > T_{\text{max}}^{(r)}) - \epsilon_{\text{slo}}] \quad (35)$$

Remote candidates must satisfy sovereignty constraints. The minimum costs are $\Phi_{\text{local}}^{r,*} = \min_{j \in \mathcal{N}_i} \Phi_{\text{local},i,j}^r$ and $\Phi_{\text{remote}}^{r,*} = \min_{i' \in C \setminus \{i\}} \Phi_{\text{remote},i'}^r$, with $i_{\text{remote}}^{r,*} = \arg \min_{i' \in C \setminus \{i\}} \Phi_{\text{remote},i'}^r$. A greedy decision selects among local execution, offloading,

and dropping (cost $V \cdot \Psi_{\text{drop}}$):

$$a_r^* = \arg \min_{a \in \{\text{local}, \text{remote}, \text{drop}\}} \{ \Phi_{\text{local}}^{r,*}, \Phi_{\text{remote}}^{r,*}, V \cdot \Psi_{\text{drop}} \}, \quad (36)$$

$$\begin{cases} \pi_{r,i} = 1, & \text{if } a_r^* = \text{local}, \\ \pi_{r,i_{\text{remote}}^{r,*}} = 1, & \text{if } a_r^* = \text{remote}, \\ \pi_{r,0} = 1, & \text{if } a_r^* = \text{drop}. \end{cases}$$

If local execution is chosen, the request is assigned to $j^* = \arg \min_{j \in \mathcal{N}_i} \Phi_{\text{local},i,j}^r$. This decentralized strategy approximates global DPP minimization with low overhead.

2) *Cost-Driven VRAM Execution Mode Selection:* After routing, each node dynamically selects the VRAM execution mode $\mathbf{M}$ based on real-time conditions. For mode m , the cost follows the DPP formulation:

$$\Phi_{i,j}^r(m) = V \cdot c_{\text{dispatch}}(r, i_{\text{src}}(r) \rightarrow i, j \mid m_{i,j}(t) = m) + Z_i(t) \cdot [\mathbb{I}(\hat{T}_{\text{TTFT}}(r, i_{\text{src}}(r), i, t, m) > T_{\text{max}}^{(r)}) - \epsilon_{\text{slo}}]. \quad (37)$$

The penalty for unserved requests is $\Phi_{\text{alt}}^r = V\Psi_{\text{drop}} + Z_i(t)(1 - \epsilon_{\text{slo}})$. The optimal mode is selected by:

$$m_{i,j}(t) = \arg \min_{m \in \{0, l^*\}} \left(\sum_{r \in \mathcal{B}_m} \Phi_{i,j}^r(m) + \sum_{r \in Q \setminus \mathcal{B}_m} \Phi_{\text{alt}}^r \right) \quad (38)$$

where l^* denotes the candidate merged LoRA selected for the node, $\mathcal{B}_m$ is the subset of queued requests feasible under mode m , and Q is the current request queue/candidate batch. As $Z_i(t)$ increases, the penalty of unserved requests dominates, prompting a switch to shared mode (Unmerged) to improve throughput and reduce latency debt.

3) *Lyapunov-Driven Auto-Scaling:* Node activation $\mathbf{N}$ is governed by Lyapunov-driven cost-benefit analysis. For node j , the net benefit is:

$$\Delta\Phi_{\text{net}}(j) = \sum_{r \in \mathcal{B}_j^*} (\Phi_{\text{alt}}^r - \Phi_{i,j}^r) - V \cdot \kappa_{i,j}^{\text{inst}} \cdot \rho_j \quad (39)$$

where $\kappa_{i,j}^{\text{inst}}$ is the unit compute cost, ρ_j the idle power factor, and $\mathcal{B}_j^*$ denotes the tentative set of requests that would be assigned to node j if it were activated. If $\Delta\Phi_{\text{net}}(j) > 0$ and resources allow, the node is activated; otherwise, it is set to standby to avoid unnecessary cost.

In summary, SP1 and SP2 jointly approximate $\mathcal{J}_{\text{Global}}$ via cross-timescale decomposition. SP1 defines a cost-efficient and sovereignty-aware placement, while SP2 applies Lyapunov-based control to handle demand dynamics without future knowledge. This closed-loop design transforms long-term QoE constraints into real-time decisions, enabling cost-efficient operation under resource and compliance constraints.

VI. EVALUATION

This section describes the experimental setup, including the testbed, system parameters, and workload models, followed by a comprehensive evaluation of the proposed cross-timescale framework using real-world traces and synthetic workloads. We compare against representative baselines and ablation variants, focusing on: (i) sensitivity to key parameters, (ii) performance under varying federation scales in terms of

TABLE III: Key Experimental Parameters and System Configuration

Parameter	Value	Parameter	Value
Model	Llama-3.1-8B	LoRA Rank	16
LoRA Cache	30 adapters	GPU	RTX 4090
$B_{\max}^{\text{merge}}$	15 requests	$B_{\max}^{\text{unmerge}}$	12 requests
τ_{pre}	0.025 s	δ_{load}	0.066 s
α_{int}	0.0012 s/req	β_{new}	0.065 s/req
TTFT SLO	6.0 s	Comp. Cost	NT\$0.001/s
Network Cost	NT\$3.0/GB	Dispatch	NT\$0.001/req
Storage Cost	NT\$0.005/GB/h	Penalty Base	NT\$0.001

throughput and latency (QoE), and (iii) cost efficiency in reducing overall OPEX under bursty and spatially skewed workloads.

A. Experiment Setup

1) *Testbed*: We evaluate both a hardware prototype and large-scale discrete-event simulations. The prototype includes a single-cluster setup with two nodes and a two-cluster deployment with one and two nodes in the two clusters, respectively. For large-scale evaluation, the simulator scales to federations with up to 3 and 10 clusters. All nodes use a single NVIDIA RTX 4090 GPU, and both environments are implemented in Python 3.10. The system operates on two timescales: SP1 (provisioning) runs hourly, while SP2 (dispatching) operates at millisecond granularity.

2) *Models*: We adopt `Meta-Llama-3.1-8B` with rank-16 LoRA adapters. Input and generation lengths are fixed at 512 and 256 tokens, respectively. Based on profiling on RTX 4090 GPUs, the profiled peak effective batch capacities are 15 in Merged Mode and 12 in Unmerged Mode. These values serve as upper bounds rather than fixed runtime batch sizes; in Unmerged Mode, the actual effective capacity is further adjusted according to the number of active LoRA adapters as modeled in Eq. (4). Each node caches up to 30 LoRA adapters in host memory.

Latency parameters are obtained via profiling. The base pre-fill latency is $\tau_{\text{pre}} = 0.025$ s, with interference and contention coefficients $\alpha_{\text{int}} = 0.0012$ s/req and $\beta_{\text{new}} = 0.065$ s/req. The cold-loading latency is $\delta_{\text{load}} = 0.066$ s, capturing disk-to-host I/O overhead.

3) *Metrics and Cost Model*: The objective is to minimize OPEX while satisfying a TTFT SLO of 6.0 s. All costs are in New Taiwan Dollars (NT\$). Compute cost is derived from RTX 4090 power consumption [32] and Taiwan electricity pricing [33], yielding about NT\$0.001/s. Network costs follow AWS pricing [34], with LoRA transfers at NT\$3.0/GB and cross-cluster dispatch charged according to token-byte traffic, equivalent to approximately NT\$0.001/request under our workload setting. Storage cost follows AWS EBS pricing [34] at NT\$0.005/GB/hour. The drop penalty is calibrated to the average compute cost per request (NT\$0.001) and prevents trivial rejection; its impact is analyzed in Section VI-D.

B. Workload Description

To capture both regional heterogeneity and temporal dynamics, we combine synthetic workloads with real-world traces and construct cluster preferences based on semantic distance. We consider a fixed pool of 100 LoRA models, including 70 global and 30 cluster-specific models. Local models are distributed across clusters according to federation size to ensure fairness. All models are embedded in a two-dimensional semantic space, where each cluster has a demand center; request probability is inversely proportional to distance, capturing spatial skewness and semantic locality.

1) *Synthetic Workload*: For synthetic workloads, target models follow a Zipf distribution with parameter $s = 1.2$, reflecting power-law demand where a few popular models dominate while long-tail models are rarely accessed. This setting induces both heavy-load and long-tail scenarios, stressing system robustness under contention.

2) *Real-world Trace Workload*: We use the Microsoft Azure Functions Trace 2021 (MAF 2021) to model temporal dynamics, preserving original arrival timestamps. These traces are mapped to cluster-specific LoRA demand rankings, enabling the simulation to jointly capture realistic burstiness and controlled regional preferences.

C. Baselines and Comparison Methods

We compare the proposed method with representative systems and ablation variants.

- **LRU + Random + S-LoRA (S-LoRA)**: adopts S-LoRA's unified paging and unmerged execution [9] as a representative single-node baseline. To enable a fair system-level comparison, we complement it with LRU caching and random dispatching to emulate multi-node operation, while it still lacks coordination and dynamic VRAM adaptation.
- **LRU + dLoRA (dLoRA)**: builds on dLoRA's hybrid (merge/unmerge) execution and intra-cluster migration [11], representing cluster-level optimization. We similarly incorporate LRU caching, while it lacks global semantic awareness and inter-cluster coordination.
- **Ours w/o Semantic (w/o-Sem)**: removes semantic-aware replacement in SP1, retaining exact matching with Lyapunov-based scheduling (SP2).
- **Ours w/o SP2 (w/o-SP2)**: retains semantic-aware deployment (SP1) but replaces SP2 with static allocation and random dispatching.
- **Ours**: integrates semantic-aware deployment (SP1) and Lyapunov-based scheduling with dynamic VRAM mode switching (SP2) for joint cost and QoE optimization.

Ours is the only method combining semantic-aware replacement, dynamic VRAM adaptation, and cross-timescale optimization.

D. Parameter Sensitivity Analysis

Before comparing different methods, we examine the sensitivity of two key control parameters in the proposed framework and determine their default settings for the remaining experiments.

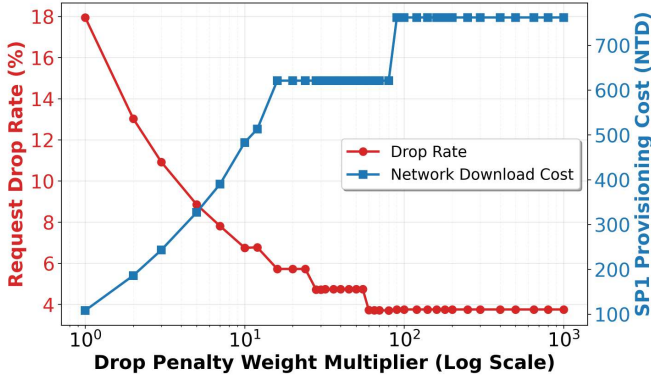


Fig. 4: Impact of the Drop Penalty Multiplier on Request Drop Rate and Provisioning Cost

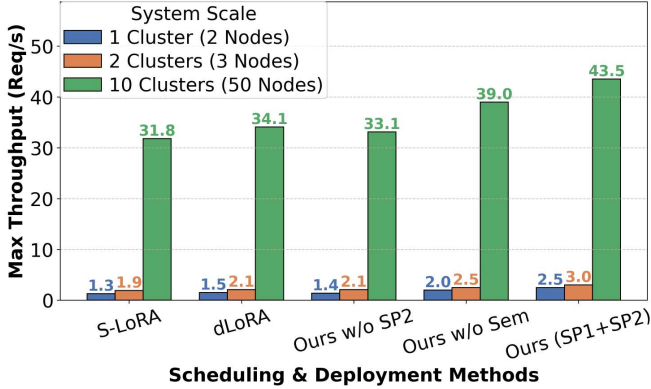


Fig. 6: Maximum Throughput Comparison Across System Scales

1) *Impact of the Drop Penalty Weight:* We first study the effect of the drop penalty weight Ψ_{drop} on system behavior. Fig. 4 shows the average operating cost and request drop rate under a high-load synthetic workload as the penalty multiplier varies. When the penalty is small, the optimizer tends to reject requests instead of activating additional resources or dispatching them across nodes, yielding low cost but an unacceptably high drop rate. This indicates that an insufficient Ψ_{drop} overemphasizes cost minimization at the expense of service availability.

As the penalty increases, the drop rate decreases rapidly, indicating that the system absorbs more traffic through node activation, execution-mode switching, and cross-cluster dispatching. However, beyond a threshold (approximately multiplier 60), the drop rate saturates at about 3.7%, while the provisioning cost reaches a plateau. Further increasing the penalty provides little additional benefit but maintains a high cost level. Therefore, we set the penalty multiplier to 60 in the remaining experiments, corresponding to $\Psi_{\text{drop}} = 60 \times 0.001 = \text{NT}\0.06 , as it provides a good trade-off between availability and cost efficiency.

2) *Trade-off of the Lyapunov Control Parameter:* We next examine the impact of the Lyapunov control parameter V on long-term cost and latency. Fig. 5 presents the time-average operating cost and P95 TTFT under the MAF 2021 workload

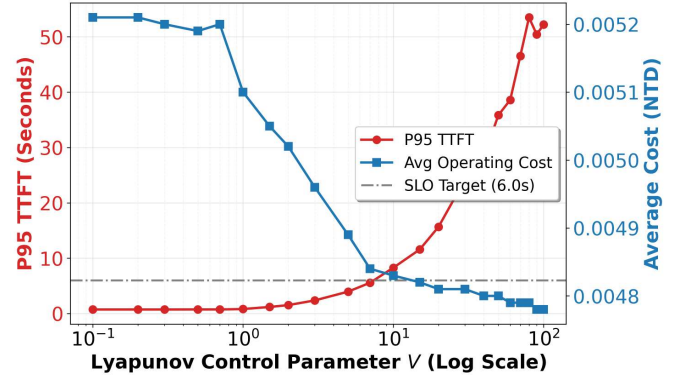


Fig. 5: Trade-off between OPEX and Latency vs. Lyapunov Parameter V .

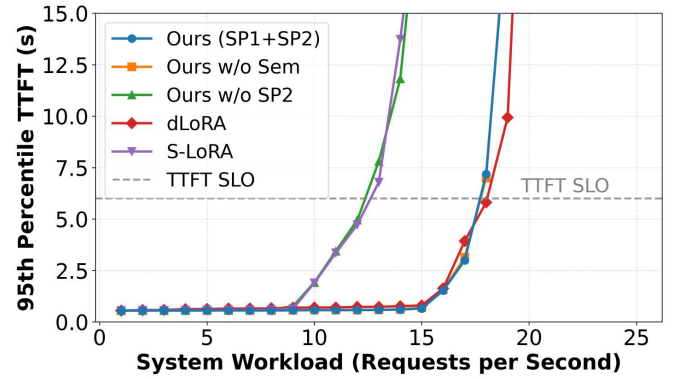


Fig. 7: P95 TTFT under Increasing Load and System Saturation

as V varies. As V increases, the average cost decreases while the P95 TTFT rises, exhibiting the standard cost-delay trade-off predicted by Lyapunov optimization. Notably, the cost reduction gradually saturates, whereas the latency continues to increase and eventually violates the SLO.

This trend follows directly from the drift-plus-penalty (DPP) formulation. A larger V places more emphasis on cost reduction, encouraging the system to save resources by delaying node activation or reducing inter-region offloading. In contrast, a smaller V makes the scheduler more responsive to virtual queue backlog and thus more willing to consume additional resources to suppress latency. Based on this trade-off, we set $V = 8$, which achieves the lowest cost while still satisfying the P95 TTFT SLO of 6.0s.

E. Throughput and TTFT Analysis

We evaluate system capacity and latency. To isolate latency from request dropping, the drop mechanism is disabled for all methods, forcing every request to be served so that true capacity can be compared.

1) *Maximum Throughput:* Fig. 6 compares the maximum steady-state throughput under different system scales. Baselines perform poorly across all settings. S-LoRA suffers from VRAM fragmentation due to random dispatching and lack of coordination. dLoRA improves single-cluster utilization via

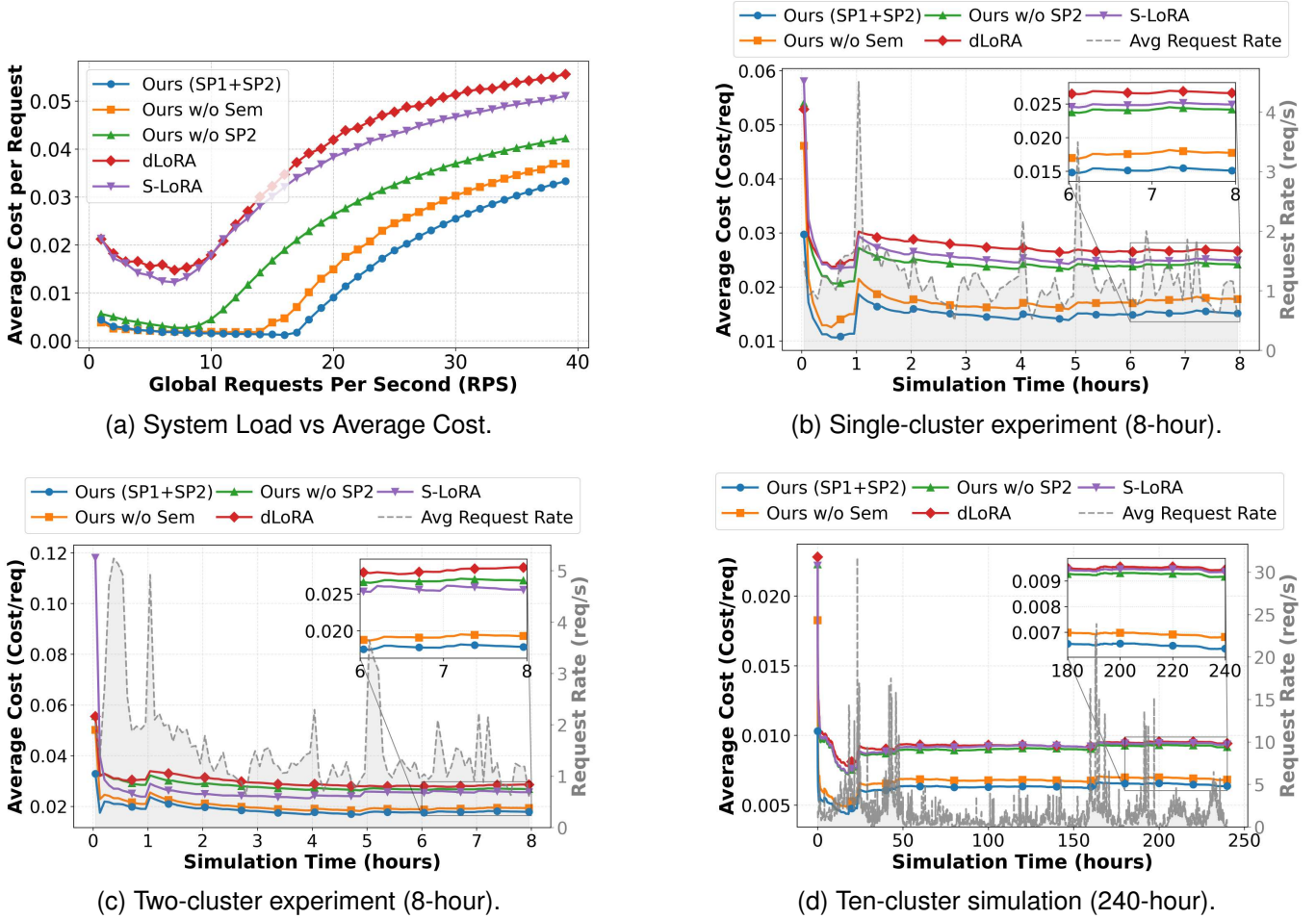


Fig. 8: System cost under varying load and long-term real-world traces.

intra-cluster migration, but gains are limited in multi-cluster scenarios due to the absence of cross-cluster consolidation.

In contrast, our method consistently achieves the highest throughput and scales effectively with system size. Ablation results confirm the importance of both components: removing SP1 leads to fragmented placement and reduced throughput, while removing SP2 degrades batching efficiency. By combining semantic-aware consolidation and dynamic execution-mode switching, the framework maximizes parallelism without additional hardware.

2) *P95 TTFT versus RPS*: Fig. 7 shows P95 TTFT as workload (RPS) increases in the 15-node setting with a 6.0 s SLO. Under light load, all methods exhibit similar latency. As load increases, S-LoRA and w/o-SP2 degrade rapidly at relatively low RPS (11–13), indicating early saturation due to queue buildup and limited adaptability.

dLoRA maintains lower tail latency under moderate load due to effective per-cluster scheduling. Our full system achieves comparable latency across most workloads and reaches a similar saturation point, indicating efficient resource utilization under balanced demand. However, removing SP2 causes significantly earlier degradation, highlighting the role of adaptive scheduling. While semantic grouping has limited

impact in this balanced setting, it introduces no overhead and becomes critical under skewed or dynamic workloads, as shown later. Although the saturation point under balanced demand is close to dLoRA in the TTFT experiment, our method achieves higher maximum throughput under broader scale settings due to better consolidation and semantic-aware placement.

F. Cost Analysis

After establishing the superior capacity and latency performance, we next evaluate the OPEX efficiency of the proposed framework under QoE constraints from both steady-state and real-world dynamic workload perspectives.

1) *Average Cost under Different RPS*: Fig. 8(a) shows the average per-request cost as the request rate (RPS) increases. Under low load, all baselines (S-LoRA, dLoRA, and w/o-SP2) incur higher costs than our system due to inefficient request placement. S-LoRA uses random dispatching, dLoRA adopts a delay-aggressive policy that spreads sparse requests across instances, and w/o-SP2 lacks dynamic scheduling for effective consolidation. These lead to poor batching efficiency and unnecessary node activation. In contrast, our method

leverages Lyapunov-driven auto-scaling to match active nodes to demand, reducing fragmentation and lowering cost.

As load increases, the cost gap widens. Baselines exhibit steep cost growth due to (i) low model hit rates causing frequent cold starts and cross-region transfers, and (ii) degraded efficiency near capacity that amplifies penalties. Removing semantic substitution (w/o-Sem) also increases cost, highlighting its role in reducing remote dependencies. Our design achieves the lowest cost across all RPS levels by combining semantic-compatible substitution with dynamic execution-mode switching, maintaining high locality and batching efficiency. As a result, it scales more gracefully, keeping per-request cost growth moderate even under heavy load.

2) *Long-term Cost Dynamics and Cost Breakdown under Real-world Trace*: To validate the adaptability of the system to temporal fluctuations and spatial skewness, we conduct multiple 8-hour experiments and a 240-hour long-term simulation based on the MAF 2021 trace. Figs. 8(b)–8(d) present the temporal evolution of per-request cost under different system scales. As shown in Fig. 8(b)–8(d), the instantaneous cost curve of Ours closely tracks traffic fluctuations, without the uncontrolled cost growth observed in dLoRA and S-LoRA, whose long-term costs eventually settle at significantly higher plateaus. A comparison across Figs. 8(b)–8(d) further shows that cross-cluster cooperation effectively absorbs regional traffic bursts and smooths the cost peaks and valleys.

The superior cost efficiency of Ours can be attributed to the joint effect of several system components. First, SP2 combines dynamic scaling with semantic-consolidated execution to reduce redundant resource activation and improve overall utilization efficiency. Second, the predictive deployment in SP1, together with in-place semantic substitution, helps avoid unnecessary cross-region model transfers, thereby reducing network overhead. Finally, under bursty traffic conditions, inter-cluster offloading in the federated architecture mitigates request drops and stabilizes system behavior. These mechanisms collectively contribute to the consistently lower and more stable cost observed in Fig. 8. Overall, the results confirm that the proposed system preserves QoE while achieving substantial long-term cost efficiency under highly dynamic workloads.

VII. CONCLUSION

This work addresses multi-tenant VRAM contention and high OPEX in edge federations by jointly optimizing cost and QoE. We propose a semantic-aware cross-timescale framework that integrates predictive multi-cluster provisioning with real-time dispatching under data-sovereignty and edge storage constraints. Leveraging behavioral embeddings, the framework enables semantic LoRA substitution across clusters, reducing redundant transfers and cold-start overhead. A Lyapunov-based controller manages VRAM modes and routing with dynamic auto-scaling, ensuring QoE guarantees while minimizing end-to-end operational cost. Extensive simulations using Azure Functions traces and synthetic workloads show that the proposed method outperforms baselines in throughput and long-term cost efficiency, while satisfying 95th-percentile TTFT

SLO under diverse workload conditions. The results suggest that semantic-aware substitution and hierarchical multi-timescale orchestration are not merely implementation-level optimizations, but necessary design principles for scalable, compliant, and cost-efficient Multi-LoRA serving in federated edge environments.

REFERENCES

- [1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, “Language models are few-shot learners,” *Advances in neural information processing systems*, vol. 33, pp. 1877–1901, 2020.
- [2] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “Autogen: Enabling next-gen LLM applications via multi-agent conversations,” in *First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=BAakY1hNKS>
- [3] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [4] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [5] Y. Chiang, Y. Zhang, H. Luo, T.-Y. Chen, G.-H. Chen, H.-T. Chen, Y.-J. Wang, H.-Y. Wei, and C.-T. Chou, “Management and orchestration of edge computing for iot: A comprehensive survey,” *IEEE Internet of Things Journal*, vol. 10, no. 16, pp. 14 307–14 331, 2023.
- [6] R. Singh and S. S. Gill, “Edge ai: a survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.
- [7] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.
- [8] Y. Chiang, C.-H. Hsu, and H.-Y. Wei, “Collaborative social-aware and qoe-driven video caching and adaptation in edge network,” *IEEE Transactions on Multimedia*, vol. 23, pp. 4311–4325, 2020.
- [9] Y. Sheng, S. Cao, D. Li, C. Hooper, N. Lee, S. Yang, C. Chou, B. Zhu, L. Zheng, K. Keutzer, J. E. Gonzalez, and I. Stoica, “S-lora: Serving thousands of concurrent lora adapters,” 2024. [Online]. Available: <https://arxiv.org/abs/2311.03285>
- [10] L. Chen, Z. Ye, Y. Wu, D. Zhuo, L. Ceze, and A. Krishnamurthy, “Punica: Multi-tenant lora serving,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.18547>
- [11] B. Wu, R. Zhu, Z. Zhang, P. Sun, X. Liu, and X. Jin, “{dLoRA}: Dynamically orchestrating requests and adapters for {LoRA}{LLM} serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 911–927.
- [12] Y. Sui, H. Wang, H. Yu, Y. Hu, J. Li, and H. Wang, “Serverlesslora: Minimizing latency and cost in serverless inference for lora-based llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.14468>
- [13] Y. Ni, X. Yang, Y. Tang, Z. Qiu, C. Wang, and T. Yuan, “Predictive-lora: A proactive and fragmentation-aware serverless inference system for llms,” in *Proceedings of the 2025 6th International Conference on Computer Science and Management Technology*, 2025, pp. 1267–1273.
- [14] H. Sfaxi, I. Lahyani, S. Yangui, and M. Torjmen, “Latency-aware and proactive service placement for edge computing,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, pp. 4243–4254, 2024.
- [15] J. Cheng, D. T. Nguyen, and V. K. Bhargava, “Resilient edge service placement under demand and node failure uncertainties,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 1, pp. 558–573, 2023.
- [16] Z. Shen, Y. He, Z. Wang, Y. Zhang, G. Sun, W. Ye, and A. Li, “Edgelora: An efficient multi-tenant llm serving system on edge devices,” in *Proceedings of the 23rd Annual International Conference on Mobile Systems, Applications and Services*, 2025, pp. 138–153.
- [17] S. Lee, S. Das, M. Gupta, and K. P. Gummadi, “Lora on the go: Instance-level dynamic lora selection and merging,” *arXiv preprint arXiv:2511.07129*, 2025.
- [18] N. Iliakopoulou, J. Stojkovic, C. Alverti, T. Xu, H. Franke, and J. Torrellas, “Chameleon: Adaptive caching and scheduling for many-adapter llm inference environments,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, 2025, pp. 217–231.

- [19] Y. Chiang, C.-H. Hsu, G.-H. Chen, and H.-Y. Wei, “Deep q-learning-based dynamic network slicing and task offloading in edge network,” *IEEE Transactions on Network and Service Management*, vol. 20, no. 1, pp. 369–384, 2022.
- [20] A. Lekharu, A. P. S. Chauhan, A. Sur, and M. Patra, “Reinforcement learning-based adaptive bitrate caching at mec server,” *IEEE Transactions on Network and Service Management*, vol. 21, no. 3, pp. 3292–3304, 2024.
- [21] N. Di Cicco, G. F. Pittalà, G. Davoli, D. Borsatti, W. Cerroni, C. Raffaelli, and M. Tornatore, “Scalable and energy-efficient service orchestration in the edge-cloud continuum with multi-objective reinforcement learning,” *IEEE Transactions on Network and Service Management*, 2025.
- [22] K. Tajiri and R. Kawahara, “Optimization of data and model transfer for federated learning to manage large-scale network,” *IEEE Transactions on Network and Service Management*, 2025.
- [23] S. Jaiswal, S. Arun, A. Parayil, A. Mallick, S. Mastorakis, A. Khare, C. Alverti, R. S. Amant, C. Bansal, V. Rühle, and J. Torrellas, “Serving heterogeneous lora adapters in distributed llm inference systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.22880>
- [24] Y. Xia, F. Fu, W. Zhang, J. Jiang, and B. CUI, “Efficient multi-task LLM quantization and serving for multiple loRA adapters,” in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. [Online]. Available: <https://openreview.net/forum?id=HfpV6u0kbX>
- [25] R. Chen, C. Yu, H. Fu, X. Hu, and B. Yang, “Mixlora: An efficient multi-tenant framework for concurrently serving diverse lora models in large language models,” in *Proceedings of the 54th International Conference on Parallel Processing*, 2025, pp. 11–21.
- [26] R. Li, H. Bao, H. Wang, P. Shi, and G. Yi, “Jointserlora: Cache-aware lora inference serving,” in *2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS)*. IEEE, 2025, pp. 1–2.
- [27] S. Li, H. Lu, T. Wu, M. Yu, Q. Weng, X. Chen, Y. Shan, B. Yuan, and W. Wang, “Toppings:{CPU-Assisted},{Rank-Aware} adapter serving for {LLM} inference,” in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 613–629.
- [28] H. Zhang, J. Shi, Y. Wang, Q. Chen, Y. Shan, and M. Guo, “Improving the serving performance of multi-lora large language models via efficient lora and kv cache management,” *arXiv preprint arXiv:2505.03756*, 2025.
- [29] T.-Y. Chen, Y. Chiang, J.-H. Wu, H.-T. Chen, C.-C. Chen, and H.-Y. Wei, “Ieee p1935 edge/fog manageability and orchestration: Standard and usage example,” in *2023 IEEE International Conference on Edge Computing and Communications (EDGE)*. IEEE, 2023, pp. 96–103.
- [30] Hugging Face, “Lora - parameter-efficient fine-tuning (peft),” https://huggingface.co/docs/peft/conceptual_guides/lora, 2026, accessed: 2026-04-16.
- [31] L. A. Barroso and U. Hölzle, “The case for energy-proportional computing,” *IEEE Computer*, vol. 40, no. 12, pp. 33–37, 2007.
- [32] ServeTheHome, “Nvidia geforce rtx 4090 founders edition review,” <https://www.servethehome.com/nvidia-geforce-rtx-4090-founders-edition-review-the-gpu/>, 2023, accessed: 2026-04-15.
- [33] Taiwan Power Company / Ministry of Economic Affairs, “Taiwan electricity pricing information,” <https://www.taipower.com.tw/>, 2025, accessed: 2026-04-15.
- [34] Amazon Web Services, “Aws pricing overview,” <https://aws.amazon.com/pricing/>, 2026, accessed: 2026-04-15.



Yao Chiang (Member, IEEE) received the B.S. and M.S. degrees in management information systems from the National Pingtung University of Science and Technology, Pingtung, Taiwan, in 2014 and 2016, respectively, and the Ph.D. degree in electrical engineering from National Taiwan University (NTU) in 2021, where he is currently a Postdoctoral Research Fellow. His research interests include data mining, machine learning and mobile communications design for multi-access edge computing systems.



Hung-Yu Wei (Fellow, IEEE) is a Distinguished Professor in the Department of Electrical Engineering and the Graduate Institute of Communications Engineering (GICE) at National Taiwan University (NTU), where he currently serves as Director of GICE. He previously served as Interim Chair and Associate Chair of the Department of Electrical Engineering and as Chair of the IoT Research Center at NTU. He received his B.S. degree in electrical engineering from NTU in 1999, and his M.S. and Ph.D. degrees in electrical engineering from Columbia University in 2001 and 2005, respectively. He was a summer intern at Telcordia Applied Research in 2000 and 2001 and was with the NEC Labs America from 2003 to 2005. His research interests include next-generation wireless networks, IoT, and fog/edge computing.

He received the K.T. Li Distinguished Young Scholar Award from the ACM Taipei/Taiwan Chapter in 2012, the Wu Ta-You Memorial Award from MOST in 2015, the Outstanding Electrical Engineering Professors Award in 2024, and the MOST/NSTC Outstanding Research Award in 2020 and 2025. He has also received the NTU Excellent Teaching Award three times. He serves as Chair of the IEEE 1935 Working Group on edge management and orchestration and is an Associate Editor of the IEEE Internet of Things Journal and IEEE IoT Magazine. He is an IEEE Fellow and an AAIA Fellow.



Guan-Lin Chen received the B.S. degree in electrical engineering from National Taiwan University (NTU), Taiwan, in 2024. He is currently pursuing the M.S. degree in electrical engineering at NTU. His research interests include edge computing and Large Language Models (LLM).